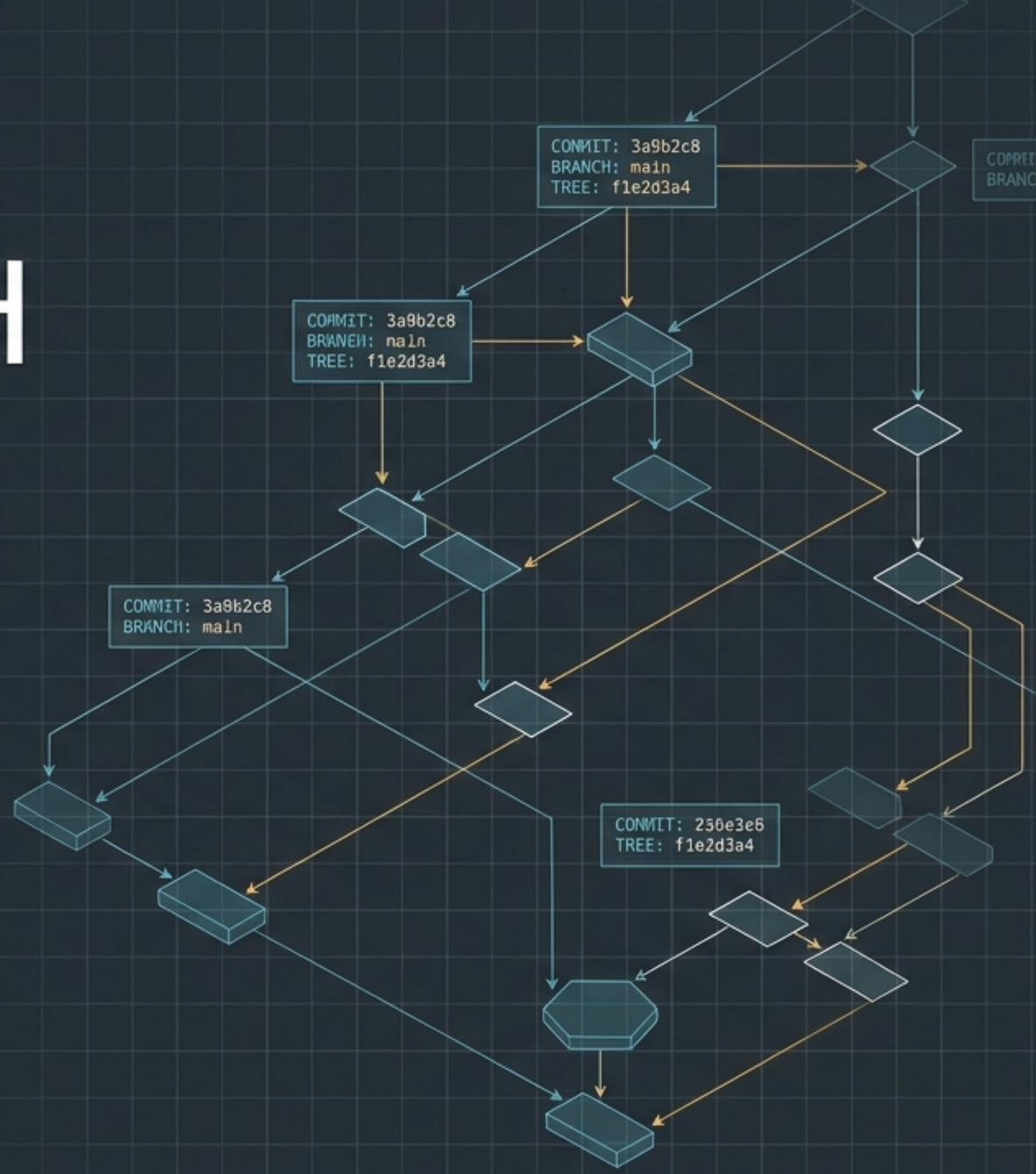
Mini Git (b5-2) 아키텍처 및 알고리즘 심층 리뷰

서드파티 라이브러리 없이 밑바닥부터 구현한 분산 버전 관리 시스템의 핵심 원리




```
mini-git> init
Initialized repository. (main -> None)
```

Gemini Notebook

코어 철학: Zero-Dependency 엔지니어링

순수 Python 표준 라이브러리 (외부 API 및 정렬 함수 전면 금지)



Gemini Notebook

Q1. 커밋 그래프는 왜 DAG이며 내부 자료구조와 불변성은 어떻게 설계되었는가?

Time Complexity: O(1) (조회)



미래를 향하는 역방향 간선 불가: 무한 루프 원천 차단

객체 불변성 (Immutability)

`__setattr__` 오버라이드를 통해 임의 속성 변경 시 `AttributeError` 발생
런타임 무결성 완벽 강제

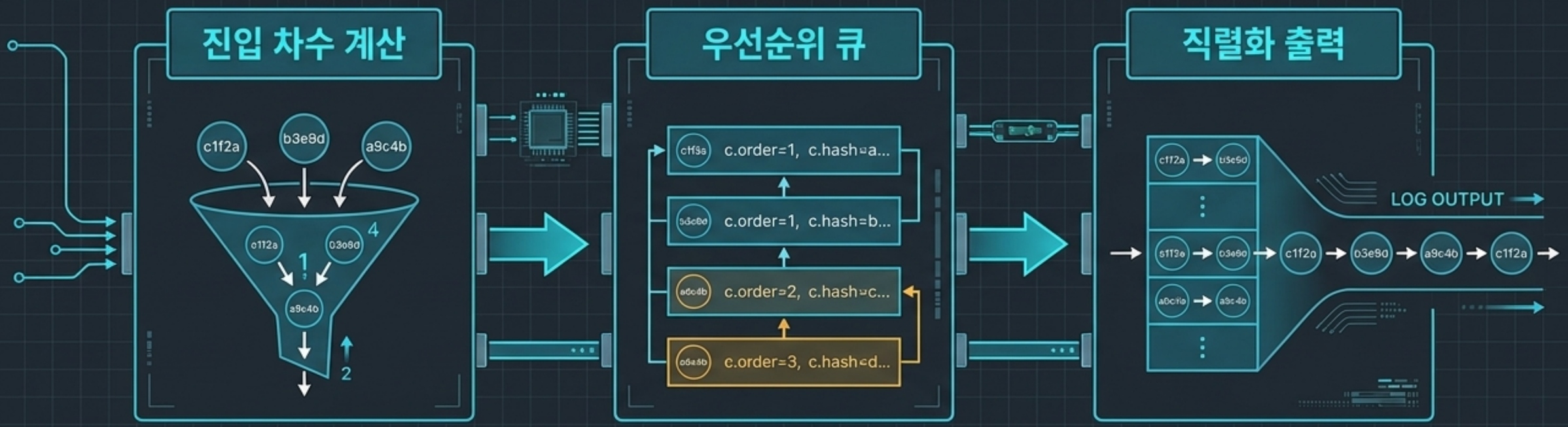
내부 저장소 메커니즘

`dict[str, Commit]` 형태의 해시맵 구조
상수 시간 `O(1)` 조회 보장

Gemini Notebook

Q2. 위상 정렬을 이용한 부모 커밋 우선 LOG 출력 원리와 타이브레이킹은 무엇인가?

O(V log V + V + E)



Kahn 알고리즘 적용:
부모 우선 출력을 위해 간선 의존성을 뒤집어 진입 차수(In-degree) 추적

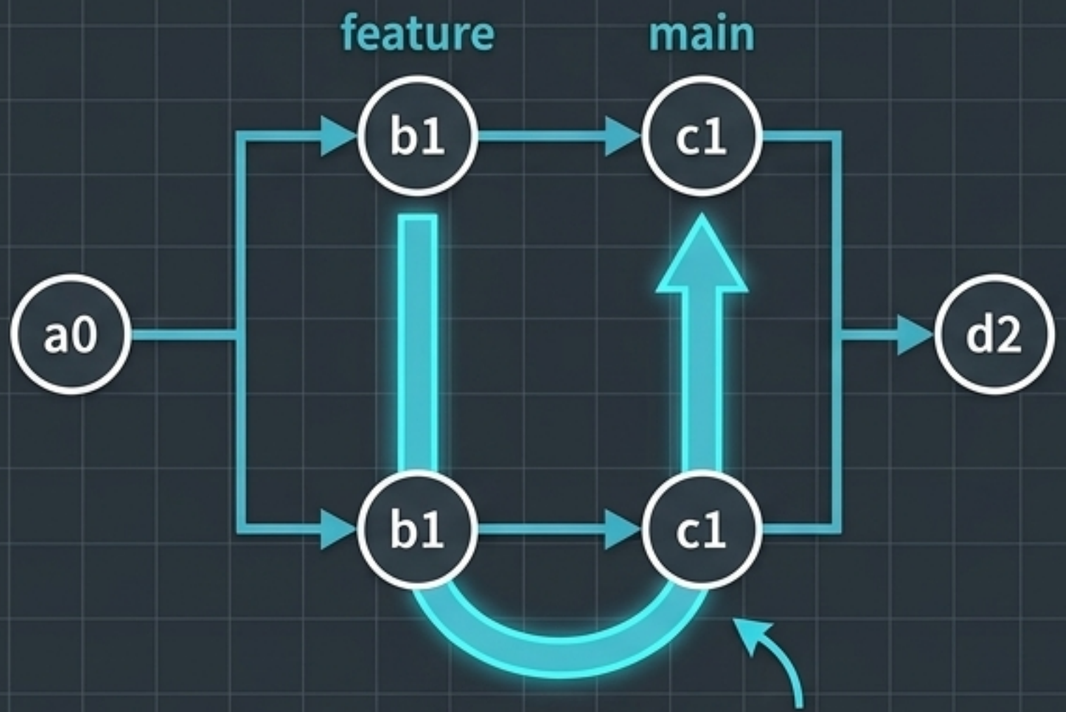
결정론적 타이브레이킹:
동순위 발생 시 (c.order, c.hash) 복합 키 기준 오름차순 안정 정렬

부모가 우선순위로 빠져나오는 직렬화된 로그 스트림

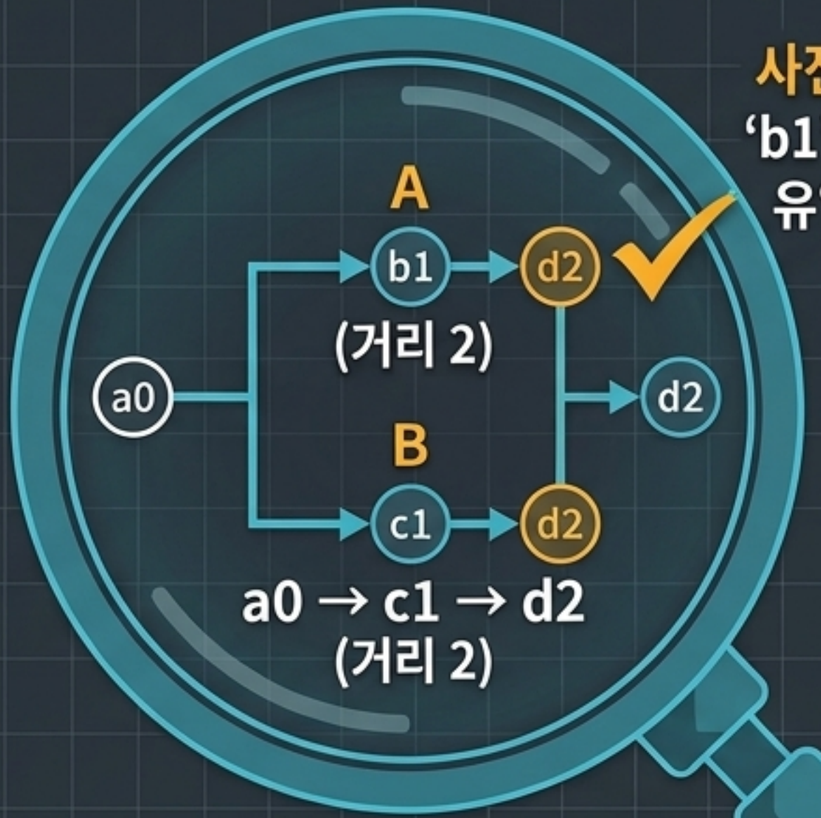
Gemini Notebook

Q3. 두 커밋 간 최단 경로 탐색에서 무방향 간선 간주 및 BFS 타이브레이킹 원리는 무엇인가?

Time Complexity: O(V + E)



무방향 확장: 서로 다른 브랜치 이동 시 공통 조상(LCA)을 거치는 양방향 탐색 필수



사전순 타이브레이크:
'b1' < 'c1' 조건에 의해 유일하고 결정론적인 최단 경로 채택

Gemini Notebook

Q4. 특정 커밋의 모든 조상 탐색 알고리즘과 출력 정렬 기준은 무엇인가?

O(V' + E')

스택 기반 DFS 역추적 탐색

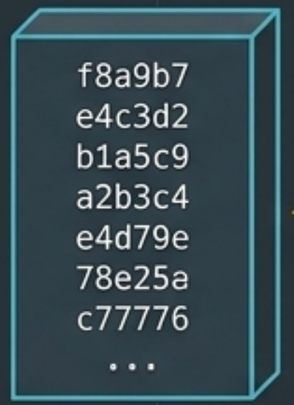


방문 세트(Visited Set)를 활용하여 다이아몬드 머지 구조의 중복 탐색 차단

메모리 구조 및 정렬 보장



Call Stack



출력 정렬 보장: DFS 반환 집합을 커밋 생성 순번(order) 기준 오름차순 안정 정렬하여 표시

Gemini Notebook

Q5. 역색인(Inverted Index) 구조, 토큰화 정책 및 메모리 최적화 원리는 무엇인가?

Time: O(1) / Space: O(N * T)

선형 탐색 (기존 방식)	역색인 룩업 (Mini Git)
	
선형 스캔 O(N)의 한계: 매 검색마다 모든 커밋 메시지 파싱으로 심각한 지연 발생	**단일 룩업 O(1) 달성:** 단어에서 해시 리스트로 즉각적 도출

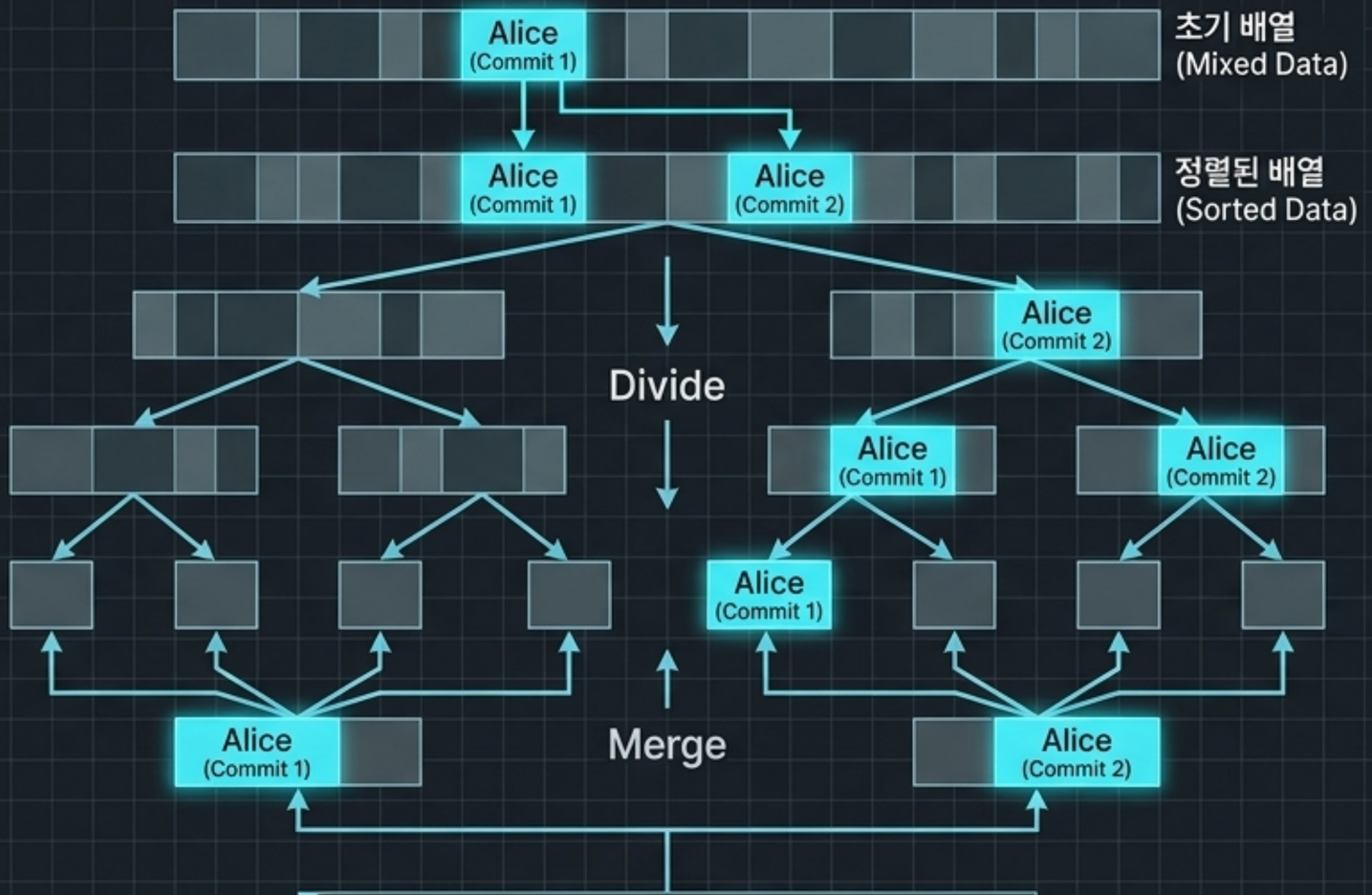
토큰화 및 공간 트레이드오프

- **기술적 구두점 보존:** `fix(auth):`, `issue#123` 등 식별자 의미 유지
- **중복 필터링:** 단일 커밋 내 중복 단어 1회만 등록
- **메모리 압축:** 4바이트 정수 ID 매핑 시 92.9% RAM 절감

Gemini Notebook

Q6. 직접 구현한 병합 정렬(Merge Sort)의 복잡도와 메모리 특성은 무엇인가?





f_x 복잡도 확정 보장

마스터 정리에 의해
최선/평균/최악
100% O(N log N) 보장

공간 트레이드오프

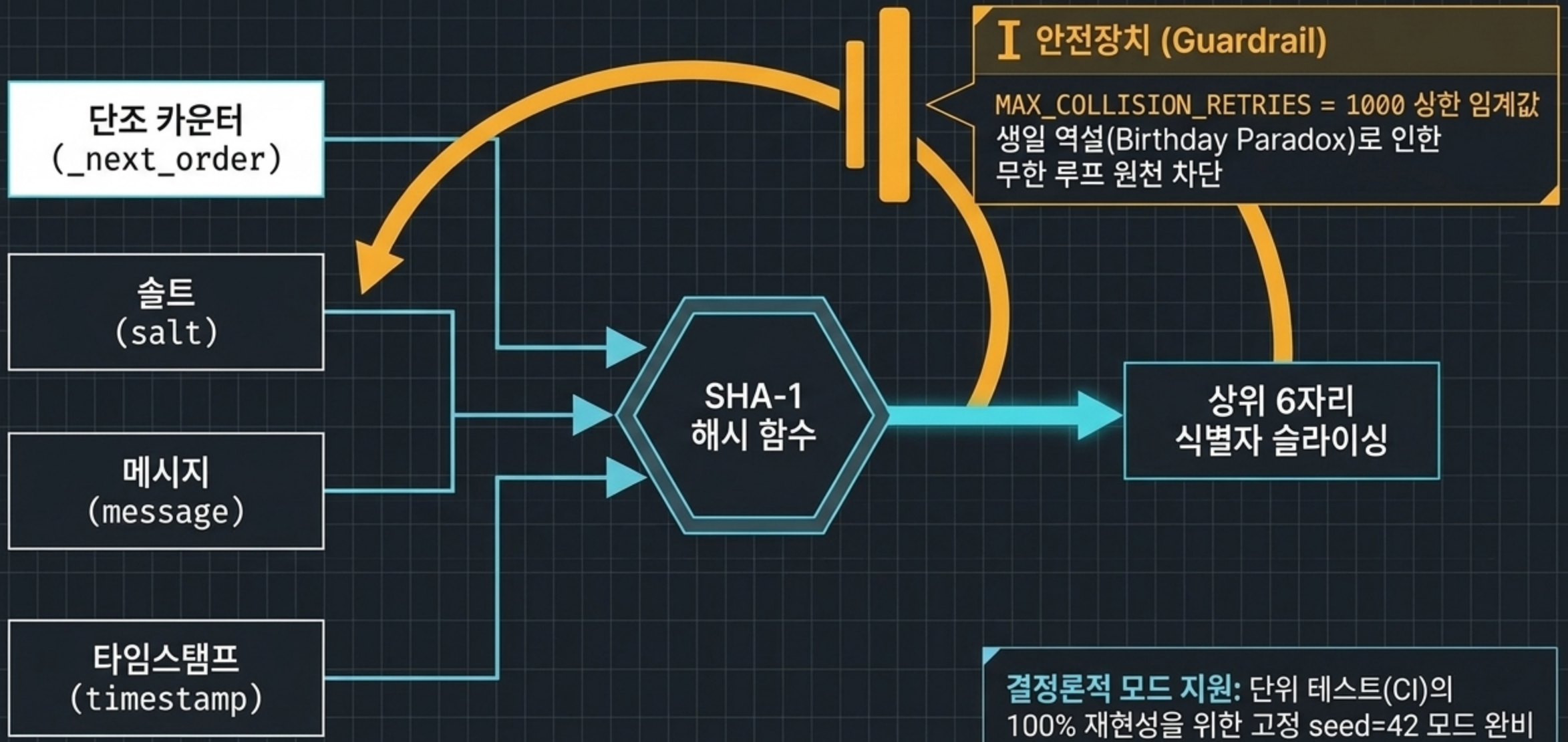
O(N) Auxiliary Space
오버헤드를 수용하여
퀵 정렬의 최악 O(N^2)
퇴화 위험 방어

안정 정렬(Stable Sort) 필수성

동일 작성자 질의 시 본래의 시간 순서(order)가
섞이지 않도록 안정성 절대적 보장

Gemini Notebook

Q7. 충돌 방어 6자리 SHA-1 해시 생성과 테스트 재현성 메커니즘은 무엇인가?



Gemini Notebook

Q8. 대규모 확장성 확보를 위한 병목 분석 및 아키텍처 고도화 전략은 무엇인가?

대규모 5,000노드 PoC 벤치마크 실측 결과

위상 정렬 지연 한계 극복

기존 (전체 Kahn 순회)

26.01ms

1,767x Faster

0.014ms ← 개선
(세대 번호 증분 갱신)

무방향 BFS 탐색 가속

매번 동적 빌드

4.59ms

2.0x Faster

2.26ms

인접 리스트 캐시 재사용

역색인 메모리 압축

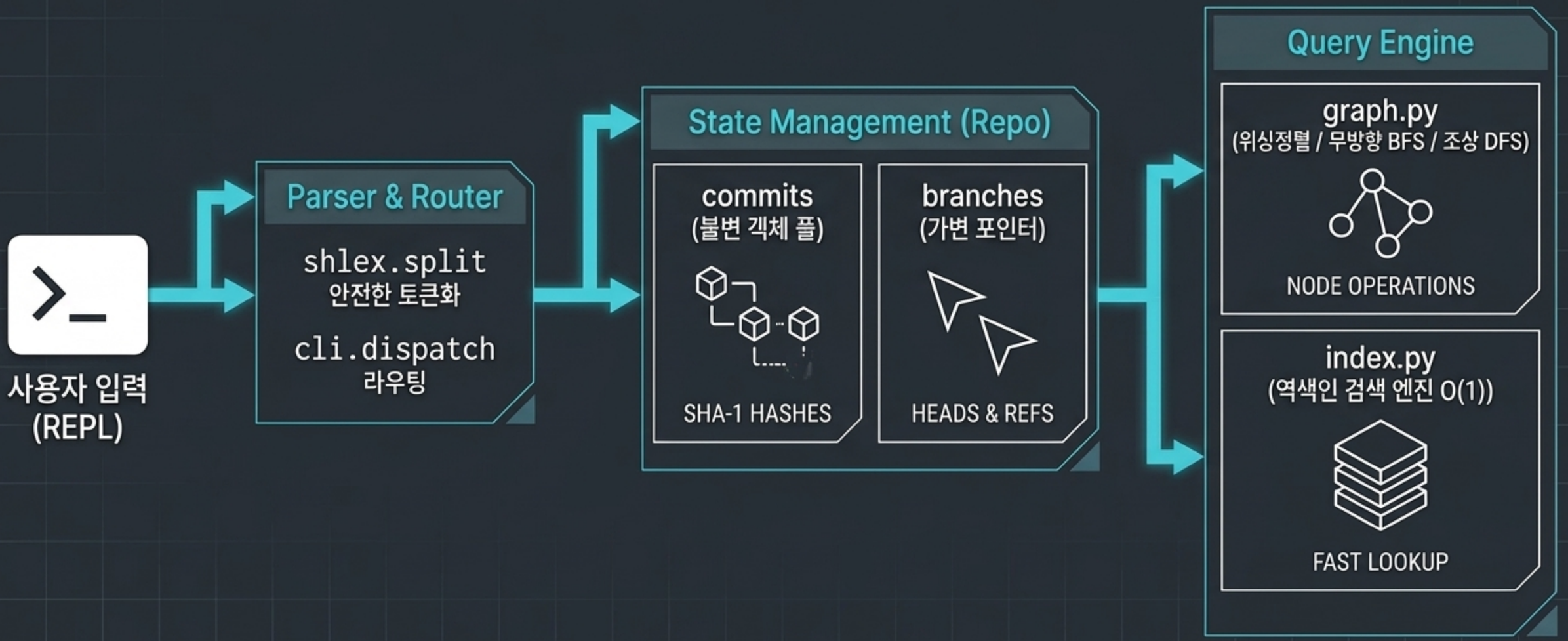
문자열 딕셔너리
1,093KB

92.9% Less RAM

4바이트 정수 ID 매핑
78KB

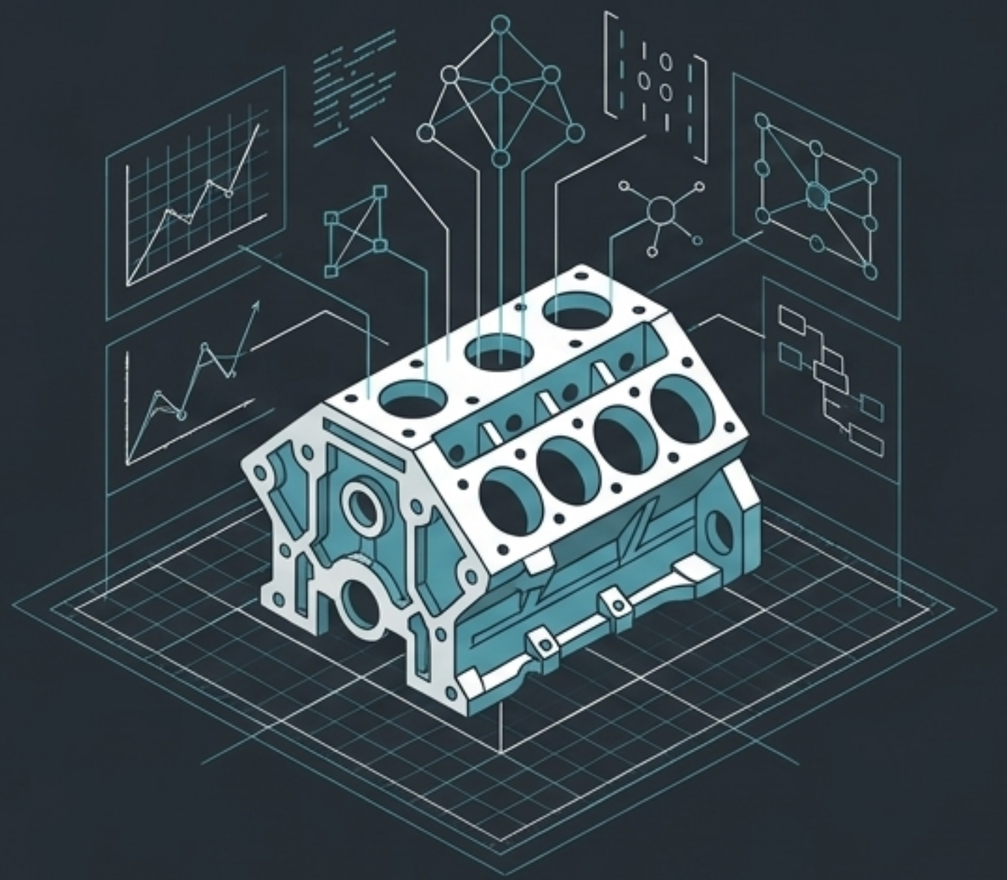
Gemini Notebook

System Architecture Flow: 사용자 명령부터 노드 저장까지



Gemini Notebook

결론 및 기술적 성취



완전한 통제력 확보

내장 정렬 API와 외부 라이브러리에 의존하지 않고,
핵심 자료구조의 밑바닥을 100% 설계 및 통제

수학적 엄밀성 검증

알고리즘 복잡도의 이론적 한계를 5,000노드 PoC 실측
성능으로 증명 및 12대 전수 단위 테스트 완벽 통과

프로덕션 수준의 안정성 방어

shlex 셸 인젝션 방어, 메모리 OOM 방어, 해시 충돌
무한 루프 상한 등 실무적 신뢰성(Reliability) 모델 완비

Gemini Notebook